J. Parallel Distrib. Comput. 74 (2014) 1775–1788

Contents lists available at ScienceDirect

J. Parallel Distrib. Comput.

journal homepage: www.elsevier.com/locate/jpdc

Efficient allocation of resources in multiple heterogeneous Wireless Sensor Networks

Wei Li [a,*], Flávia C. Delicato [b], Paulo F. Pires [b], Young Choon Lee [a], Albert Y. Zomaya [a], Claudio Miceli [b], Luci Pirmez [b]

[a] Centre for Distributed and High Performance Computing, School of Information Technologies, The University of Sydney, NSW 2006, Australia
[b] DCC/IM, Federal University of Rio de Janeiro, RJ 20001-970, Brazil

HIGHLIGHTS

- The proposed algorithm is designed to effectively exploit the heterogeneity of WSNs.
- The proposed algorithm is designed to enable running multiple applications simultaneously.
- SACHSEN is implemented as a functional component of an existent middleware.

ARTICLE INFO

Article history:
Received 29 January 2013
Received in revised form
8 August 2013
Accepted 26 September 2013
Available online 7 October 2013

Keywords:
Wireless sensor network
Resource allocation
Task allocation

ABSTRACT

Wireless Sensor Networks (WSNs) are useful for a wide range of applications, from different domains. Recently, new features and design trends have emerged in the WSN field, making those networks appealing not only to the scientific community but also to the industry. One such trend is the running different applications on heterogeneous sensor nodes deployed in multiple WSNs in order to better exploit the expensive physical network infrastructure. Another trend deals with the capability of accessing sensor generated data from the Web, fitting WSNs in novel paradigms of Internet of Things (IoT) and Web of Things (WoT). Using well-known and broadly accepted Web standards and protocols enables the interoperation of heterogeneous WSNs and the integration of their data with other Web resources, in order to provide the final user with value-added information and applications. Such emergent scenarios where multiple networks and applications interoperate to meet high level requirements of the user will pose several changes in the design and execution of WSN systems. One of these challenges regards the fact that applications will probably compete for the resources offered by the underlying sensor nodes through the Web. Thus, it is crucial to design mechanisms that effectively and dynamically coordinate the sharing of the available resources to optimize resource utilization while meeting application requirements. However, it is likely that Quality of Service (QoS) requirements of different applications cannot be simultaneously met, while efficiently sharing the scarce networks resources, thus bringing the need of managing an inherent tradeoff. In this paper, we argue that a middleware platform is required to manage heterogeneous WSNs and efficiently share their resources while satisfying user needs in the emergent scenarios of WoT. Such middleware should provide several services to control running application as well as to distribute and coordinate nodes in the execution of submitted sensing tasks in an energy-efficient and QoS-enabled way. As part of the middleware provided services we present the Resource Allocation in Heterogeneous WSNs (SACHSEN) algorithm. SACHSEN is a new resource allocation heuristic for systems composed of heterogeneous WSNs that effectively deals with the tradeoff between possibly conflicting QoS requirements and exploits heterogeneity of multiple WSNs.

Crown Copyright © 2013 Published by Elsevier Inc. All rights reserved.

1. Introduction

Wireless Sensor Networks (WSNs) represent is a domain of distributed computing that has attracted great research interest in recent years. A WSN is composed of a large number of battery-operated devices equipped with one or more sensing units, processor, memory, and wireless radio. The main goal of WSNs is to collect data about physical phenomena and send the necessary data back to users through one or more exit points called sink nodes. Sink nodes are more powerful devices, often a personal computer, that are in charge of gathering the collected sensing data, further processing them, and make them available to WSN

* Corresponding author.
E-mail address: liwei@it.usyd.edu.au (W. Li).

0743-7315/$ – see front matter Crown Copyright © 2013 Published by Elsevier Inc. All rights reserved.
http://dx.doi.org/10.1016/j.jpdc.2013.09.012

applications. WSNs enable the observation and monitoring of a large geographic area, often inhospitable and/or remote places, in a timely and accurate manner. Such capability has motivated the employment of WSNs in a wide range of applications, such as structural monitoring, habitat, wildlife and environmental monitoring, surveillance systems, and location tracking. Each kind of application has different requirements in terms of type of sensing data, delivery model, data sample rate, as well as different QoS requirements.

The first generation of WSNs adopted application-specific designs, with the primary goal of achieving energy efficiency. Most of these WSNs were designed for running a single target application, even for large-scale implementation. This limited use of numerous versatile sensor nodes could be the major source of system inefficiency, even for the purpose of providing redundancy. Besides resource wastage, such design approach does not fully exploit the WSN capabilities of self-adapting, self-configuring and autonomy that enable the same nodes to perform different tasks. Currently, industry-driven WSNs targeting diverse applications are increasingly being designed and deployed. These emergent WSN applications are normally required to collect and process data from different types of sensor nodes with different QoS requirements, and the sensor nodes can be manufactured by various vendors and implemented using different programming language and models. The issues of heterogeneity of the sensor nodes, different application requirements, high distribution and various network structures impose extra challenges to the WSN design and make the implementation and execution of WSN applications non-trivial. Consequently, the new generation of WSN system design is essentially required to integrate heterogeneous WSNs to perform multiple (possibly) co-existing WSN applications, meeting distinct requirements, within such a system which possibly has overlapping physical space between the WSNs. This design trend is well represented by some existing industrial paradigms, such as: Internet of Things [1], Web of Things [13] and cyber-physical systems [18]. An obvious but key challenge faced by such kinds of systems is how to allow users to fully utilize the resources of heterogeneous sensor nodes to complete the WSN applications successfully without knowing the details of underlying system infrastructure while still remaining energy efficiency.

A promising approach to meet the design and implementation issues of running multiple WSN applications on the system consists in incorporating a middleware platform in the heterogeneous WSNs. There are several approaches for WSN middleware with different purposes and design approaches [23]. However, all WSN middleware in their core should aim to provide the same capabilities to integrate and well utilize system resources on demand and to help abstracting away the distribution and heterogeneity of the underlying computing environment. A middleware solution which is able to provide high flexibility for adding important and sophisticated functions to WSN is the service-oriented approach. In Service-Oriented Middleware (SOM), sensor nodes are considered as service providers for WSN applications. Abstractions for the complex underlying WSN are provided as set of services that can be accessed by WSN applications though standardized interfaces based on the service-oriented architecture (SOA) [23]. The atomic services of SOM are mainly originated from the three basic operations performed by sensor nodes (sensing, data processing and communication) and these atomic services form more generic services, such as task allocation, node selection, resource discovery, data fusion, security, or application specific services, such as FFT computing, and other mathematic functions. All services can be designed, implemented and integrated in SOM to provide a flexible and easy environment to develop effective WSN applications. This paper focuses on developing a heuristic task allocation algorithm as a component of SOM to distribute WSN applications to sensor nodes in an adaptive, energy-efficient and application-transparent manner, while meeting the different application QoS requirements.

In this context, we present the *reSource AlloCation in Heterogeneous SEnsor Networks* (SACHSEN) algorithm as a new resource allocation heuristic to address the challenges faced by running multiple applications onto heterogeneous WSNs. SACHSEN is the core of a resource allocation component of a service-oriented WSN middleware. We adopted an existent middleware [9] and extended it with SACHSEN implementation, by exploiting the inherent capability of flexibility and extensibility provided by the middleware modular and service oriented design approach. SACHSEN effectively exploits the heterogeneity and application diversity aiming to maximize the system lifetime while meeting minimum performance requirements of applications. In comparison with most resource allocation approaches in WSN, that only consider a single application at a time, SACHSEN is developed to allocate applications (including simultaneous arrivals) to sensor nodes based on the application utility which is decided by selected QoS requirements, as well as fully exploiting the system heterogeneity. The QoS requirements in WSNs can be specified from two perspectives [5], application QoS and network QoS. In this paper, we use data accuracy (application QoS) and system lifetime (network QoS) as the selected metrics to calculate application utility since they are strongly related to the entire system performance. The rationale behind SACHSEN is that in the process of resource allocation, it can exploit the fact that different applications may share the same sensing data provided that the data they are interested in have common characteristics in time and space, and similar QoS requirements. In such a case, a new application will not necessarily incur an increase of the usage of the WSN resources.

The remainder of this paper is organized as follows. Section 2 discusses previous work related to this study. The overview of the system architecture, as well as the adopted middleware is introduced in Section 3. Section 4 describes the application, system and energy models. The proposed algorithm SACHSEN is presented in details in Section 5. In Section 6, we evaluate the performance of SACHSEN followed by conclusion and future work in Section 7.

2. Related work

Recently, much emphasis has been given to task-sensor assignment in WSNs explicitly taking into account energy efficiency. Although a number of efforts in this respect have been made, these efforts are not directly applicable to the scenario considered in this paper, i.e., several QoS-aware applications running on top of multiple WSNs endowed with heterogeneous resources. Probably, the problem of allocation or scheduling of multiple applications in distributed and heterogeneous computing systems, such as clusters and grids is similar to our scenario. Specifically, the authors in [25] proposed several resource allocation algorithms (for a homogeneous multi-cluster system) that judiciously constrain the amount of resources that can be allocated to each incoming application. Algorithms proposed in [16] first combine multiple task graphs into a single composite task graph, which is then scheduled using a standard task graph scheduling algorithm. Afterwards, another two heuristic algorithms are employed to perform task-by-task scheduling over all tasks in all task graphs to optimize fairness. A three-tiered system model is adopted in [4] to address the issue of dynamically scheduling applications on a wide-area network computing system. This paper presents a two-level distributed scheduling algorithm for multiple tasks graphs on the aforementioned environment. The first level is a WAN-wide distributed scheduler responsible for dispatching different applications to the second level schedulers. Each second level scheduler is LAN-wide and centralized, but has no knowledge of other LANs. It

is fully responsible for generating the scheduling scheme for the allocated tasks. No matter what algorithm is proposed for distributed computing, the first priority is to minimize the makespan, but not energy-efficiency or other WSN related performance metrics. Furthermore, the resource allocation problem addressed in other areas (e.g. grid, cluster and wireless network) are, however, fundamentally different from that addressed in WSNs. For example, cluster and grid computing systems address issues like number of machines and task completion time, while the wireless network mostly addresses network-level issues like data rate, packet delay, throughput and packet-loss-probability. Therefore, those algorithms developed for allocating tasks onto multi-clusters systems cannot be directly applied to the multi-WSNs system or even traditional WSNs.

Several noticeable solutions for allocating tasks onto a single (homogeneous) WSN with different selection objectives have also been proposed. In [3], Byers and Nasser introduced a solution modeling the sensor-task assignment problem with notions of utility (accuracy of the collected data and its usefulness to a task) and cost (the energy consumption of activating, operating the sensors and any other possible energy consumption associated with selection). They aimed to develop an algorithm that maximizes the utility while adhering to a predefined (energy) budget. A market-based sensor-task assignment method is proposed in [24], where the tasks are allocated to those sensors which provide information or "goods" which can be purchased while observing certain budgets. Another energy-aware sensor-task assignment algorithm [27] is proposed by Rowaihy et al. using joint utility functions to obtain the near-optimal allocation. In their solution, energy efficiency is achieved by evenly distributing the tasks among sensors. However, in their proposal, all the tasks are associated with different priorities, and each sensor is only able to process one task at a time. In [28], the authors adopted a WSN system similar to the one proposed in this paper, but the focus of their work is to find out the best tradeoff between fidelity-aware resource allocation and the sensor node selection to support multiple concurrent applications. Fidelity is defined as an application-dependent concept that can denote a variety of operational measures including communication latency, data quality and redundancy. In order to support more applications running in the WSN system simultaneously, the algorithm reduces the value of fidelity of several tasks in the applications to meet the requirement, which induces the quality reduction for that application. SACHSEN achieves efficiency in performance and energy usage without such reduction. Overall, the works in [3,24,27,28] can be considered as a multi-objective resource allocation problem in WSNs and more detailed information can be found in [26]. Besides the above solutions, service-oriented computing (SOC) has started to be used for integrating heterogeneous WSNs to perform multiple applications recently. In [12], the adaptive Servilla, which is a kind of SOM presented in [21], is proposed to allocate tasks onto heterogeneous WSNs subject to the energy efficiency and network dynamic. It treats services as the basic unit of software modularity so that they can be dynamically provided to different applications. The adaptive Servilla not only provides SOC entirely within WSNs, like ∈ SOA [30] and OASiS [17], but also process the service binding online. Another interesting SOC implementation is presented in [7], which enables multiple applications outside of the WSNs to efficiently share resources within WSN. Their solution uses SOC to reuse and reserve WSN resources to maximize the quality of data provided to each user, but it does not take the energy efficiency or other WSN-related QoS metrics into account.

The problem we tackle here is distinct from previous works since it considers the tradeoff of multiple conflicting QoS parameters between data accuracy, energy efficiency and load balancing as primary target, as well as it proposes an energy-efficient technique of data sharing for tasks from different applications, in a heterogeneous WSNs scenario. We employ the energy-saving task allocation technique as our solution to prolong the overall system lifetime while meeting the aforementioned requirements. Until fairly recently, we noticed that a recent work [19] presented by Li et al. shares the same concerns and strives for the similar goals of this work. Even though these two works have several aspects in common, [19] is developed as a standalone multi-phase approach for addressing the needs of performing multiple time-constraint applications on the top of a cluster-based hierarchical WSNs system, whereas this work is a first attempt to develop a one-phase approach integrating with middleware to enable WSN applications with different QoS requirements simultaneously running across multiple WSNs. Compared to [19], SACHSEN also has wider adaptability to the target systems, which are composed of multiple heterogeneous WSNs that are owned, managed and maintained by multiple administrative authorities with potentially competing economic interests.

3. Overview of system architecture and the adopted SOM middleware

This section introduces the general concept of SOM and the framework of the adopted SOM middleware. Before presenting those details, we first give an overview of the system architecture for better understanding our study scenario.

3.1. System architecture

The general heterogeneous WSNs system considered in this work encompasses several WSNs, each one with a different number (and types) of sensor nodes. These WSNs are loosely connected by a communication network and independently managed and accessed by different sink nodes. Each WSN may have the common sensing area with other WSNs. The system architecture is shown in Fig. 1, where three major elements, namely web server, sink nodes and sensor nodes are organized in a hierarchical manner.

In the system, the web server acts as a frontier to handle dynamic arrivals of application, to decompose an application into tasks, decide how to dispatch the incoming tasks to WSNs based on certain criteria and gather the updated WSNs information. From an external point of view, final users and external applications request different services from the system by sending tasks; and the web server acts as a service provider to reply those requests. By tasks, we mean the units of execution that consist of an application submitted by users. More precisely, the user submitted applications often have specific QoS requirements and they require the respective services (e.g., detect fires, monitor temperatures) provided by the system to be completed. As such, distribution of the necessary work over the heterogeneous WSNs can be transparent to the application that requires the related services from the system. Given an application that is divisible into multiple tasks, different allocation of these tasks on the nodes of the system will consume different amounts of energy, as well as obtain different quality of services. After receiving tasks from the web server, the sink node of each WSN schedules these tasks to individual sensor nodes, called task executors, which are the entities that effectively perform the task processing. From an internal point of view, sink nodes are the service requesters and sensor nodes are the service providers. Sensor nodes send the descriptions of their services to sink nodes, which keep a repository of the service descriptors of each type of existing sensors in their respective networks. Whenever a task is completed, the sensor node responsible for its execution sends the result back to the sink node. The sink in its turn forwards the results to the requested client application, via the web server.

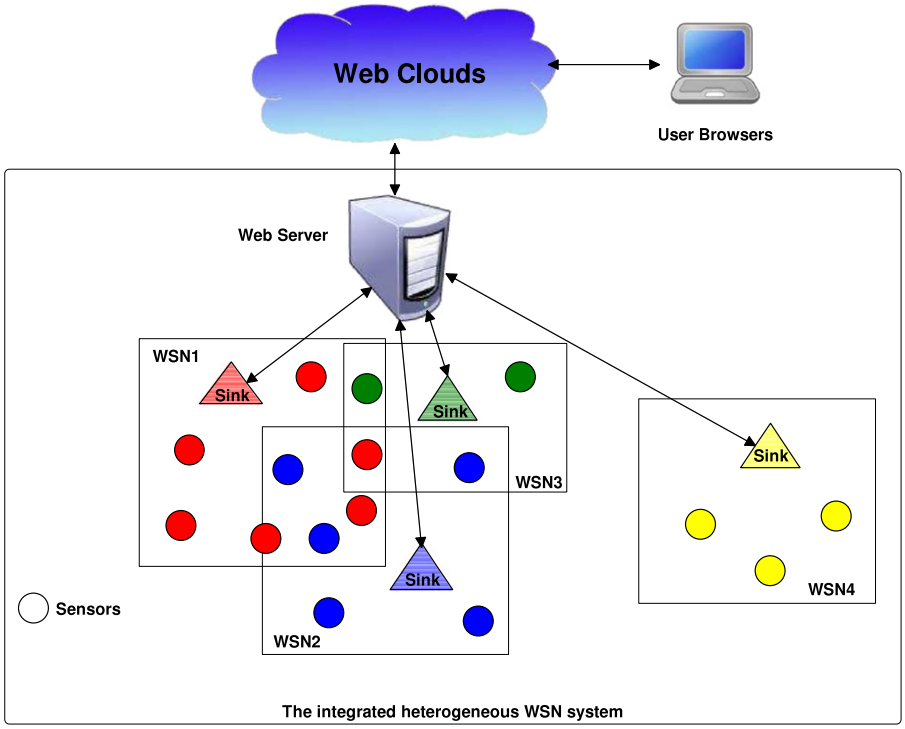


Fig. 1. The system architecture of heterogeneous WSNs.

3.2. Middleware architecture

In traditional distributed systems, a middleware seeks primarily to hide the complexity of the underlying networking environment by insulating applications from dealing with explicit protocol handling, data replication, network faults, and parallelism. In the same way, a middleware for WSN has to insulate applications from the particular features of the underlying infrastructure and network protocols. It has to include mechanisms to formulate complex high-level sensing tasks, communicating those tasks to the WSN and coordinating sensor nodes to perform the tasks. It has also to guarantee the efficient usage of the available resources. Besides, it must be robust, fault tolerant, and lightweight to meet the scarce resources typical of sensor nodes.

Several WSN middleware have been proposed in the past years. In our work we will adopt an existent middleware, called MARINE, described in [9], and enhance its functionalities by adding the proposed task scheduling algorithm. MARINE (*MiddlewAre for Resource and mIssion-oriented sensor Networks*) is a middleware specifically designed for WSN, based on a component model [33] and built on two architectural patterns, namely microkernel [29] and REST [11]. The adoption of a component model simplifies the programming task, promoting decoupling and modularization of system functionality in well-defined components, and facilitates the management of the middleware, providing the separation of the system responsibilities and features into discrete units. As a potential advantage of such design approach, a high degree of customization and extensibility is achieved. Therefore, the middleware can be tailored to the specific application(s) to execute in the WSN at a given moment, and it can also be easily extended with additional components, such as the task allocation algorithm proposed in this paper. Besides adopting a component model, MARINE is based on the microkernel pattern. The microkernel can be defined as a component that encapsulates the core of the system and starts up the middleware. Further services are initialized and removed according to the needs of the specific application(s) running in each node. These design features are desirable in the WSN context, for saving resources of sensor nodes. Finally, in order to facilitate the access to the WSN and promote interoperability among diverse networks, MARINE adopts a service-oriented approach, based on the REST pattern. REST defines a lightweight communication between applications based on Web standards. Using a REST-based approach WSN data can be accessed as Web resources, providing an abstraction layer that hides from client users and/or applications the peculiarities of WSN hardware devices and software platforms besides allowing the integration of data collected by other Web systems. Furthermore, by providing a high level and standardized interface for data access, MARINE promotes interoperability of networks from different technologies, thus leveraging the current trend of building heterogeneous systems involving multiple networks and applications.

The main purpose of adopting the microkernel pattern in MARINE is to make the middleware core as small as possible to ensure a minimal memory consumption of the WSN nodes. During the middleware boot process, MARINE executes only a minimal core. After booting this core, the remainder required components for executing a particular application(s) are selectively booted, allowing the load of a customized middleware instance. The microkernel starts the middleware basic services that need to remain active (i.e. loaded in the main memory and running) throughout the middleware lifecycle, enabling the delivery of the sensor-collected information to client applications. These basic services are implemented as component Managers in the middleware architecture. Additional managers and components are initialized only if they are necessary to meet the requirements of running applications. The other components remain inactive until eventually be required by an arriving application. The middleware encompasses a set of data repositories, each one with a manager in charge of controlling the use of the respective repository, acting as interfaces between their data and the other components.

In its original version, MARINE offers a REST-based communication service as well as adaptation and configuration services. New services can be specified by third parties and incorporated by using MARINE component model and the provided APIs. Status information about MARINE components is obtained through the

occurrence of predefined events and is made available to the application whenever it is necessary or desired. The *configuration service* configures sensor nodes according to requirements of a given mission aiming at satisfying such requirements by selecting the most suitable communication protocols, duty cycle of nodes and logic topology to be adopted in the network.

In order to facilitate control and running WSN applications, MARINE provides a component called *Application Manager*, responsible for managing the lifecycle of applications installed on the node. The function of such manager is initializing, pausing, resuming and destroying applications stored in the *Application Manager Repository*. The *Application Manager* allows MARINE managing all installed applications as a whole, or individually, thus assisting the task of managing such applications. The *Application Manager* is also responsible for configuring MARINE according to the application requirements. By accessing the *Application Repository*, the *Application Manager* obtains the requirements of each application. Through the requirements, the *Application Manager* starts all the requested middleware services (components).

In our adaptation of MARINE to accommodate the proposed algorithm, SACHSEN, a different set of components is loaded into each different type of node (web server, sink and sensor node). Besides using MARINE existing functionalities to accommodate SACHSEN requirements, a new service was specified. The *resource management service* was implemented as a new component, called ResourceManager, incorporated to the middleware architecture to be deployed in sink nodes and web servers. Such component has a set of modules and is in charge of coordinating the decomposition of application in tasks and distributing tasks to sensors upon the execution of the algorithm. By adopting the REST-based approach, different sink nodes communicate among themselves and with the web server by exchanging HTTP messages. MARINE was implemented in the Sun Spot sensor platform [6]. It is important to note that, in spite of being agnostic to a specific WSN platform, a requirement to implement MARINE is that the platform supports dynamic load of components.

4. Models and problem formulation

This section is divided into four subsections, where we first detail the models which are used in our proposal, including: application model, system model and energy model. Afterwards, the problem formulation is formally given to describe the core problem addressed in this paper.

4.1. Application model

In our proposal, an application is defined as a 4-tuple $\langle V, G, \Delta t, C\rangle$, where V denotes the tasks and each task represents a required service. G is the geographic area, determined by four geographic coordinates that define the boundary of an interested area which the application is required to monitor or sense. Δt is used to specify how long the application should last, and C represents the confidence of an application regarding its successful completion. In the context of our work, task and service are strongly correlated. A service indicates the action that can be performed by a sensor node so as to be requested for accomplishing the relevant task. A task $v_i \in V$ has the following properties: $\langle$*service_type*, *SsR*, *SdR*, *workload*$\rangle$. The *service_type* element denotes the type of the service a task requests. Each task is associated with one and only one service type (for instance, temperature, light intensity, humidity, acceleration, etc.). *SsR* and *SdR* are two user-defined elements to characterize the service provided by a sensing device. *SsR* denotes the data sensing rate, meaning the time interval between consecutive data readings the service requires to be suitably performed. *SdR* denotes the data sending rate to indicate the frequency of data sending during the time of service execution. The *workload* element denotes how long a task is to be in execution inside the WSN.

The WSN applications considered in this paper consist of one or more tasks; thus an application may require a composition of services. Specifically, each of these tasks is the finest-grained and non-divisible element to constitute the WSN application. In this paper, we assume that tasks are heterogeneous and independent from each other. We also assume that once a task is initiated it cannot be interrupted until its completion (non-preemptive task). The heterogeneity of tasks has two meanings. First, each task v_i can be executed by a specific (often different) type of sensor node. Secondly, tasks performed by different sensor nodes have different data accuracy values affected by the environmental noise which will be introduced in the next section. Moreover, different compositions of services provide a different level of confidence to the application. To the best of our knowledge, the concept of application confidence is first introduced in this paper and it regards the level of confidence that can be assigned to a particular sensor or set of sensors when performing a task to achieve an application goal. As an example, let us consider an application of fire detection. Such application can be met by different compositions of services, encompassing the following sensing tasks: CO_2, CO, temperature and smoke. In this example, CO_2, and CO detection are the services used to indicate the concentration of carbon dioxide and carbon monoxide at the monitored area. Temperature monitoring denotes the service of sensing the temperature at the monitored area and comparing the collected value with a threshold. Smoke detection is the service used to indicate whether dense smoke is detected. Each service is performed by the respective sensor. A fire can be confirmed by the outcome of an individual service, with a low level of certainty or by a composition of two or more of these services, with a higher level of certainty. Such degree of "certainty" provided by a given service or composition of services is characterized by C in the application tuple, where C stands for confidence to indicate the certainty of the application is completed successfully. The values of C range from [0%, 100%]. If the composition contains all the fire related data detected by every possible type of sensor, we have the maximum certainty or confidence (100% in this case); thus totally meeting the application needs. All the other compositions only provide a partial meeting of the application needs. In this paper, we stipulate that all applications have to be completed with the maximum certainty/confidence.

4.2. System model

We assume that there are k WSNs in the system denoted by the set $W = \{W_1, W_2, \ldots, W_k\}$, deployed into the interest (monitored) area G to ensure that any event can be detected by at least one sensor node. Each WSN W_i has a single sink node SN_i and a number of sensor nodes P. Sink SN_i is used as a gateway between the WSNs and external networks (such as the Internet) via web server. It dispatches tasks to the selected sensor nodes which are able to provide required services in that WSN. Sink maintains a data repository that describes the capabilities (services), the current states (for instance, residual energy) and the location information of each sensor node. Upon the completion of services, the sink is in charge of communicating the results to the WSN applications. In addition, each sink SN_i is also responsible for the communication with others sinks to exchange data/intermediate result whenever this is required. Any WSN could overlap a set of other WSNs. This indicates that some regions of the monitored area G are mutually covered by multiple WSNs. Thus, a certain service can be provided by the sensor nodes from different WSNs. For any given sensor node p_i^k in P, where i denotes the index of WSN and k denotes the index of sensor nodes in that WSN, it belongs to one and

only one WSN. A sensor node p_i^k is capable of providing one or more services depending on their capabilities to collect/sample different types of data, for instance, temperature, light, smoke, movement. Moreover, sensor nodes can detect all events occurred within their sensing ranges. We assume that all the sensor nodes in a given WSN W_i have a valid communication path to reach its respective sink SN_i. We also assume that all sensor nodes, although heterogeneous regarding their provided types of services (sensing units) are endowed with the same radio interface as well as communication and sensing ranges. Communication interference between sensor nodes is not considered in this paper.

In order to examine the service availability of a sensor node, we introduce a judging function that indicates whether a sensor node can provide the required service at the specified area. The judging function is shown below,

$$A(s, x, y, i) = \begin{cases} 1, & \text{if a sensor is available} \\ 0, & \text{if a sensor is unavailable} \end{cases} \quad (1)$$

where s is the service that an application requires, x and y are the location information for the monitoring event and i is the index of the sensor.

Once we consider the possibility of overlapping WSNs, there is a chance that more than one sensor node is able to provide the same service in the specific area, but with different values of data accuracy. Data accuracy in WSNs is affected by several factors, such as: nominal precision of the sensing device, the environmental noise of its measurements, the number of neighboring nodes and proximity to the target area [8]. In this paper, we only take the environmental noise of a sensor measurement (M_i) into account. The parameter M_i is mainly affected by physical characteristics of the location where the sensor node is deployed. Examples of factors that affect the sensor readings are electric signals [14], local temperature and humidity [32]. The parameter M_i is a normalized value that depends on the actual level of environmental noise N_i, where N_i ranges from 0 to 100. Real values of environmental noise are detected by the sensor node upon the reading; the values adopted in our work are relative values, utilized for purposes of comparison, so that sensor nodes providing less noisy measurements take precedence in the allocation process. The measurement of environmental noise is then given by the following equation

$$M_i = 1 - N_i/100. \quad (2)$$

4.3. Energy model

Sensor nodes in general have three functional modules [15]: communication, sensing and processing. The state of a sensor node can be described by the combined states of all individual modules. We assume that each functional module has only two states, namely *active* and *inactive*. Each module can only stay in one of two states and its status does not affect other functional modules. In this paper, the sensing and communication modules are the major functional parts we are concerned with. Thus, the energy consumption of a sensor node is a function of the node state and the time during which the node remains in that state. When the states are determined, a sensor node consumes a fixed energy rate in that time period. When the state of the corresponding functional module of a sensor node is active, the energy consumption is much greater than when it is inactive. Assuming that an insignificant amount of energy is consumed when the corresponding function module of a sensor node stays inactive, we consider that the energy consumption in that state is 0. Overall, the energy consumption of a sensor node during time t is calculated by $E(t) = E_c + E_s$, where E_c represents the energy consumption of communication module, and E_s represents the energy consumption of sensing module.

For the ease of analysis, we assume that the data exchange between two neighboring sensor nodes (within one-hop communication range) belonging to the same WSN is done though direct communication. The energy consumption of transmitting l-bit data over distance d are defined as $E_{tx}(l, d)$:

$$E_{tx}(l, d) = E_{elec} \times l + \varepsilon_{amp} \times l \times d^2 \quad (3)$$

where E_{elec} and ε_{amp} are hardware-related parameters [34]. We also assume that the receiver does not consume energy in the data exchange process. For any two distance sensors (outside one-hop communication range, still belong to the same WSN), the data communication is transferred by using shortest path based multihop routing protocol (please note that the routing process is out of scope of our work). The energy consumption of transmitting l-bit data from source to the destination is defined as $E_{tx}(l, src, des)$:

$$E_{tx}(l, src, des) = \sum_{i=1}^{k} E_{tx}(l, d) \quad (4)$$

where k is the minimum hop count the data travels from source to destination and $E_{tx}(l, d)$ remains the same meaning as shown in Eq. (3). The energy consumption of sensing module is calculated by a linear equation:

$$E_{s_i} = ER_{s_i} \times t_{s_i} \quad (5)$$

where ER_{s_i} represents energy consumption of service i in one time unit and t_{s_i} represents the time period for performing service i.

4.4. Problem formulation

The goal of the task allocation is to find out a mapping for each application on a set of sensor nodes to maximize the heterogeneous WSN system lifetime subject to application requirements. According to [10], there are several different network lifetime definitions for WSNs. In this paper, we adopted the definition of the time in which the first sensor node has its energy completely depleted. The reason for using this definition is that the death of a sensor node has a chance to cause the network to be disconnected from the system, or some areas requiring monitoring to be uncovered. Thus, the system lifetime of the heterogeneous WSN system is determined by the minimum network lifetime of a WSN in the system, which is also the minimum sensor lifetime in that WSN. We use notations $L_{p_i^k}, L_{W_i}, L_{SYS}$ to represent the lifetime of a sensor p_i^k, a WSN W_i and the entire system, respectively, and the relationship among these three notations are $L_{p_i^k} \geq L_{W_i} \geq L_{SYS}$.

For given m WSN applications, where each application A_i is composed of n tasks and a system topology O, we need to find a mapping $\Pi : V \rightarrow P$, such that each task $v \in V$ is allocated to a sensor node, $p \in P$, in the heterogeneous WSN system for execution. Let $U^x = \{U_1^x, U_2^x, \ldots, U_n^x\}$ denote a task mapping scheme of the application on the given system topology O, where x is the index of task mapping solution. Each element $U_i^x \in U^x$ is a tuple of the form $\{v_i, p_j, E_{v_i,p_j}, D_{v_i,p_j}\}$, where p_j represents the sensor to which a task v_i ($v_i \in A_i$) is allocated, E_{v_i,p_j} represents the total energy consumption (consider both energy consumption of sensing and communication) of task v_i on sensor node p_j, and D_{v_i,p_j} represents the data accuracy that can be obtained if task v_i is allocated to sensor p_j. The design objective of SACHSEN is to find a task mapping scheme to maximize the system lifetime subject to application requirements.

$$\text{Find } U^o = \arg Max(Min(L_{p_j})) \quad (6)$$

$$\text{subject to } \psi_{A_i} = f(E_{A_i}, D_{A_i})$$

where ψ_{A_i} represents the requirement of application A_i and it is a cost function of energy consumption E_{A_i} and data accuracy D_{A_i} of that application, which is detailed in the next section. As mentioned before, the task allocation problem is shown to be an NP-Hard problem, thus heuristic algorithm is employed to solve this problem in polynomial time.

5. Algorithm design

In this section, we present our SACHSEN algorithm for efficient resource allocation in heterogeneous WSNs system composed of a set of WSNs. SACHSEN is a one-phase algorithm, which means that the sensor selection and task allocation decisions are made in an integrated manner. SACHSEN is also designed for providing different options for different user requirements, as well as energy conservation for performing user requested services by:

- taking an integrated approach for allocating system resources to tasks that can exploit detailed knowledge of both application characteristics and resource availability,
- minimizing sensing cost by maximizing the data reuse of common services requested at a certain time period,
- reducing communication cost by allocating tasks to the sensor which is closest to the sink node,
- using user-defined values to supervise the algorithm to select services best suited for the user.

The SACHSEN pseudo code is shown in Fig. 2 and detailed in the next subsections.

5.1. Service analysis and candidate sensors identification

SACHSEN starts with a locality-aware service discovery. When a new application arrives, the ResourceManager middleware component installed on the web server will analyze the composition of the application. Several important pieces of information that will help to generate primary task-sensor assignment is extracted from this service analysis process including the number of tasks, requested service type of each task and the geographical location of interest. One critical step in this stage is to decompose a newly arrival application into multiple tasks for further processing. We apply the divide-and-conquer strategy by first dividing an application into a set of general service categories and then find out the finest granularity of services for each category. For each application, it would request variety types of services from the WSNs, but they can be generally classified into three categories, sensing, computation and communication according to the functionalities of sensor nodes. In the sensing category, the WSNs can provide various services, e.g. collect continuous data reading from specific sensor nodes, detect occasional event within an interested area, cooperate to perform distributed sensing tasks etc. In the computation category, the WSNs can perform the requested services like fuse incoming data temporally or spatially, make local decision, eliminate noise or redundant data, store/forward the data to the next hop and etc.…For the last one, communication category, which provide different types of data communication services among sensor nodes in the WSNs. This kind of service can be further referred to as gaining access to the medium, claiming the usage, duration and bandwidth of selected wireless communication channel, determining the routes to the destination, and creating and maintaining distributed data structures. Once the service category is determined, the well-known recursive decomposition [38] will be used in each category to divide a generic service into more specific services until all the decomposed services cannot be further divided. For each task in an application, all possible sensor nodes capable of providing the type of service the task requires are identified by Eq. (1) through sinks. Affected by its competing applications, surrounding environment and the current node status, the quality of service provided by each node could be significantly different. As mentioned in Section 4.1, each requested service from an application is normally associated with its quality demand, e.g. SsR for sensing task. The nodes cannot provide the satisfactory services are not selected as candidates for the next-step processing. In addition, in traditional Web services, the incoming task (user-request) can be randomly distributed to a service provider which is capable of providing the corresponding service. However, due to the limitation of sensor hardware, each sensor node is constrained by its sensing range and only the physical phenomenon within that range can be detected. As a result, those sensor nodes are filtered, by considering locations of those available sensor nodes, their sensing ranges and the location of requested task, and the filtered sensor nodes are all put into a candidate sensor list.

5.2. Best candidate selection

Typically, the usage of WSNs differs in several respects concerning the required quality of the provide service. Some applications demand for high data accuracy or low delay, some require the network has a long lifetime to monitor long lifecycle phenomena, and others may need both of these requirements. Therefore, an application can choose to prioritize lifetime in favor of accuracy or delay, or vice versa. In this paper we specifically consider applications that are sensitive to the accuracy of the sensor-generated data and to the energy consumption. Considering this premise and based on the classification of applications, there are three possible user requirement profiles as listed below:

- Data accuracy or precision.
- Network lifetime.
- The balance between 1 and 2 (tradeoff between energy consumption and data accuracy).

Such profiles are defined in MARINE *Application Repository*, and are used to properly configure the operation of SACHSEN.

Data accuracy

The data accuracy is one of the metrics that SACHSEN focuses on providing to the application in order to handle dynamic application arrivals and various requirements. It is an application related metric that refers to a notion of quality of provided data. SACHSEN allows users to specify, as an application's requirement, different data accuracy levels for their completion. This is an important requirement for several types of WSN applications, as for instance, object tracking and security monitoring. For this type of application, the duration (lifetime) of the WSN is not as important as the accurate delivery of the sensing data. As mentioned before, the data accuracy in our model is strongly related to the environmental noise. The less environmental noise a sensor node is affected, the higher degree data accuracy that node can provide. According to this relevance measure, the application requirement (data accuracy) is easily converted to the metric (environmental noise) about sensor node. Consequently, the sensor in the primary sensors list which has the smallest value of *Mi* is selected as the best candidate to perform the task.

Energy conservation

In contrast to applications that give preference to the accurate measurements provided by the WSN, there are application domains that require a long duration of the WSN, as for instance, habitat [22], wildlife [20] and environmental monitoring [36]. As it was previously mentioned, the energy consumption of a sensor is calculated from two parts, sensing and communication. A poor choice of the best candidate affects the quality of the resulting allocation, especially for the lifetime-based profile. To derive a lower energy consumption schedule for an application, SACHSEN selects the best candidate by considering two aspects: (1) maximize the sensing data sharing and (2) reduce the communication hops to reach the sink. The goal of choosing the best candidate from candidate sensor list is to find out a task-sensor allocation that reduces energy consumption the most and this eventually contributes to extending system lifetime.

First, from Eq. (5), we know the energy consumption of sensing device is related to two parameters, ER_{s_i} and t_{s_i}. Once the service

Input: an application, system topology, application requirements
Output: the optimal task-sensor assignment

```
//Service analysis and candidate sensor identification
When a new WSN application Ai arrives, decomposes the application Ai into n tasks, where
V=(v1, v2, … , vn)and put them into a queue Q
While size(Q) > 0 do
    For a given task vi∈V, check the service availability of sensors by using equation (1)
    Create a candidate sensors list CSi and put the sensor nodes with value 1 into the list
    Remove the task vi from queue Q
End While
//Best candidate selection
For each candidate sensors list CSi
    For each pj∈CSi do
        check the application requirements' profiles
        If profile = Precision-based do
            select the sensor with minimum Mi
        Else If profile = Lifetime-based do
            select the sensor provided minimum energy consumption
        Else If profile = Ratio-based do
            select the sensor with the closest value to the result generated by equation (7)
        End If
    End For
    Return the best sensor pj for vi
End For
The latest information of WSNs which perform the allocated tasks of application Ai is collected by
their sink nodes and send back to application planner for next arrival application
```

Fig. 2. SACHSEN pseudo code.

type is determined, ER_{s_i} is obtained and remains invariable during the execution period of task v_i. The way to reduce the energy consumption of a sensing task is to decrease the time it takes. One possible solution is to decrease the *SsR* rate of a sensing task, and let the related sensing device stay in active mode for less time. The energy consumption of sensing device is reduced correspondingly. However, this solution has a risk to sacrifice the performance of application since some critical data can be missed in the time period we force the sensing device to be idle. To address this problem, we are motivated by the idea of session persistence from [35], and use the concept to develop an energy-saving solution without lowering the *SsR*. The rationale behind the solution is to maximize the intersection of execution time of the same service from different applications. For example, suppose that two sensor nodes p_1 and p_2 are able to provide the same service S from a monitored area. A service S requested from application A_1 is assigned to the sensor p_1. When a new application A_2 arrives, it contains the same service request for the same area. At the same time, the service S of application A_1 is still running on S_1. Instead of initializing a new sensor p_2 to perform the task, we allocate this task to the sensor p_1 which is already providing the same service to application A_1. Before the service S of application A_1 stops running, the collected data can be used for both application A_1 and A_2. In the mean time, only one measurement of energy is drawn from the system, and the longer intersection time for the applications, the more energy the system saves.

Second, communication is another energy consumer for the task execution. Based on Eqs. (3) and (4), the energy consumption of communication is affected by two variables, data amount and hop count. The amount of data l is determined by *SdR*, which is a pre-defined value for each service request. Attempt to change this value can cause degradation in the quality of service. For this reason, the other variable hop count is used to reduce the energy consumption of task communication. For all tasks, the destination of communication is the sink. Since different WSNs have different topology, the sensor nodes belonging to different WSNs naturally have different number of routing hops to the sink (in here, we only consider the shortest path). According to Eq. (4), for a given communication task, its energy consumption is directly proportional to the number of hops it travels.

The purpose of energy conservation is to extend the system lifetime, while the system can only fulfill its purpose as long as it is considered "alive". Overall, SACHSEN selects the sensor node which consumes the least total energy consumption (sensing and communication) from the candidate sensor list.

Normalized data accuracy and energy consumption

As the performance of SACHSEN relies on data accuracy as well as energy consumption, we derive a formula to support flexible tradeoff between those two QoS requirements of an application, and quantitatively select the best candidate from a candidate sensor list based on the given tradeoff. Since data accuracy and energy consumption represent different performance metrics, the normalization technique is employed to generate the following equation:

$$\alpha \frac{D_{s_i}}{D_{best}} + \beta \frac{E_{residual} - E_{s_i}}{E_{residual}} \tag{7}$$

where α and β are user-defined coefficients for the normalized data accuracy and energy consumption, respectively. And these two coefficients have a relationship $\beta = 1 - \alpha$. For a given task, D_{best} represents the best data accuracy in the primary sensors list and $E_{residual}$ represents the residual energy of the WSN which the selected sensor belongs to. D_{s_i} denotes the data accuracy value for any sensor in the primary sensor list and its range from 0 to D_{best}.

E_{s_i} denotes the energy consumption value for any sensor in the primary sensor list and its range from E_{best} to ∞. Since the E_{best} is generated by a greedy search method, the possibility of the same sensor node is frequently selected could be high, thus the energy of such sensor node can be overused and the system lifetime will be shorten. Hence, the best sensor node for energy-conservation (local optimum) may not be the best choice for the purposes of system lifetime extension (global optimum) at each task allocation. To alleviate this issue, as shown in the second term of Eq. (7), SACHSEN employs a concise look-ahead mechanism. The look-ahead mechanism allows SACHSEN to select a sensor node that is not the minimum energy consumer, but the one which contributes most to prolonging the system lifetime considering a particular task and system state at the time. After this time, the sensor-task assignment with the minimum energy consumption is selected and committed.

Eventually, after the normalization, both normalized data accuracy and energy consumption range from 0 to 1; SACHSEN selects the sensor which has largest value calculated from Eq. (7).

6. Performance analysis

To evaluate the performance of SACHSEN algorithm, we first built a simulator in Java and tested it under the context tailored for multiple, heterogeneous WSNs. The simulation goal is to assess the major system performance measures: system lifetime and data accuracy; for different user requirements under different settings. After that, we performed an evaluation of the proposed algorithm implemented on real sensor nodes (SunSPOT platform [6]). These experiments evaluate the performance of SACHSEN under a realistic scenario with different kinds of applications.

6.1. Simulations

6.1.1. Parameters setup

In our simulator, there are several parameters that need to be properly addressed in the application, system and energy models. For the application model, the applications arrive to the Web server following a Poisson distribution with a mean of 10 time units. At each arrival time, there is a chance more than one application arrives, and this is controlled by a parameter called multiple application arrival rate that ranges from 0 to 1 with the default value 0.1. The larger the value multiple application arrival rate has, the higher possibility of multiple applications will occur at each arrival time. The maximum number of applications arrives to the Web server at the same time is set to 3. Each arriving application contains 2 to 6 independent tasks. The average workload of a task is randomly generated from a uniform distribution ranging from 5 to 15. The SsR is set to ∞ for all tasks, which means during the execution time of the task, the related sensing device needs to monitor the area of interest at all time. SdR ranges from 1 to 3, and the total amount of communication data is not greater than the collected sensing data for completing the specific task.

For the system model, we built a WoT scenario which has one Web server, 2 to 10 WSNs and each WSN contains a varying number of sensors ranging from 2 to 50. The number of sensing devices of each sensor is uniformly distributed within the range of 1 and 5. Within a WSN, all the sensors have the same sensing range and communication range, but are subject to different environmental noises. The WSN topology has two types, single-hop and multi-hop. The multi-hop count ranges from 2 to the maximum number of sensors in a WSN. Each WSN overlaps one or more WSNs with range [0%, 100%] and the default overlapping value is set to 50%.

For the energy model, according to [34], the energy-related parameters in Eq. (3) is set as $E_{elec} = 50$ nJ/b and $\varepsilon_{amp} = 10$ pJ/b/m^2.

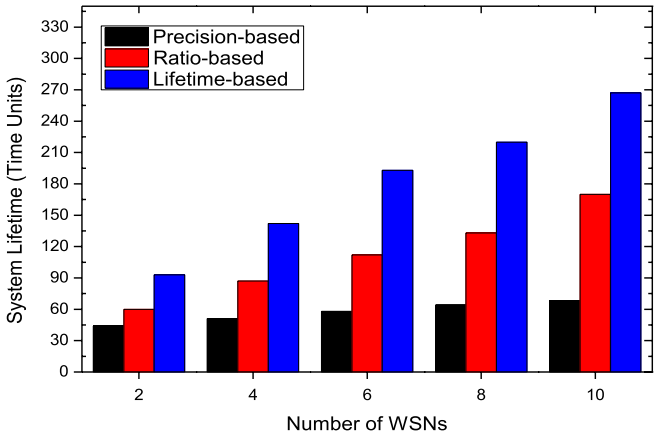


Fig. 3. System lifetime for user-requirements profiles with different number of WSNs.

Please note that, in this simulation, the communication energy is only consumed by the sender. Considering that in WSNs communication energy consumption is often greater than sensing energy consumption, we set the parameter ER_{s_i} as 10 nJ/t in Eq. (4). The initial energy for all the sensors in the simulation is set to 1.0×10^{-5} J. In addition, for the ratio-based profile, we set the same value for the coefficients α and β in Eq. (5), thus both of them are equal to 0.5.

In the following, the system lifetime, data accuracy and energy consumption per task are used as the main metrics for the performance evaluation. Unless specifically stated, all the aforementioned parameter values will be used as default setting in the experiments. All the presented results were averaged over at least 300 times, consequently, a 95% confidence interval with a 5% (or better) precision is achieved.

6.1.2. Results

In the next subsections we present results based on the three profiles specified in Section 5.2.

6.1.2.1. System lifetime. System lifetime is probably the most concerned metric for the WSNs performance evaluation. In such a resource-constrained environment, the every bit of limited energy consumption must be considered. System lifetime as a measure for energy consumption occupies the exceptional position that it consists in an upper bound for the utility of the WSNs. Therefore, we first investigate how system lifetime is influenced by the user requirements' profiles.

Results shown in Fig. 3 indicate that, with the increase in the number of WSNs encompassed in the heterogeneous WSNs system, the system lifetime when using the precision-based profile does not significantly change; however the lifetime-based profile obtains the most benefit to extend the WSN lifetime. This is because, under the precision-based profile, SACHSEN will always select the sensor which has the lowest environmental noise to perform the tasks. As a consequence, those tasks requesting the same service from the same area will be always allocated to the same sensor (s). Apparently, by adopting this allocation strategy a sensor will be easily overdrawn, and the system lifetime strongly depends on the lifetimes of the single nodes that constitute the network. As a result, precision-based profile performs poorly when the number of WSNs increases. Instead, the lifetime-based approach will always seek for the least energy consumption for the tasks. When the number of WSNs increases, SACHSEN has more sensors to perform the tasks due to the overlapping of WSNs and thus its use prolongs the system lifetime. The ratio-based

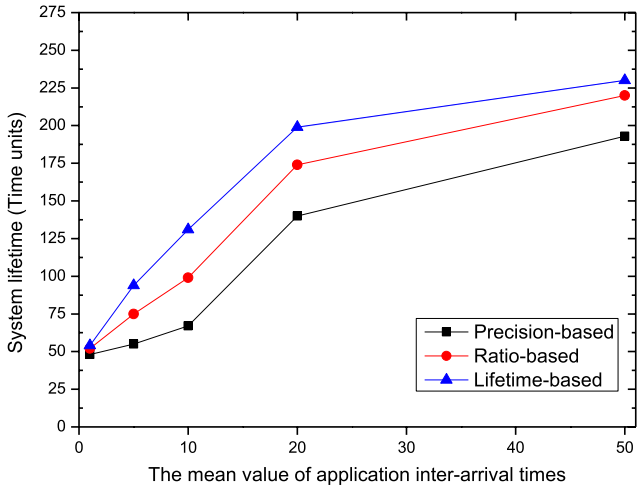


Fig. 4. System lifetime for user-requirements profiles with different application arrival rate.

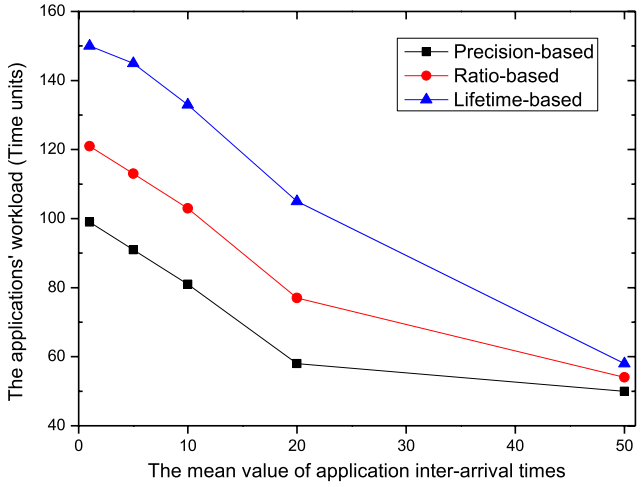


Fig. 5. Total application time for user-requirements profiles with different application arrival rate.

profile gets the characteristics from both profiles, so it has medium performance among the three profiles.

We also investigated the impact of the mean applications arrival time interval over the system lifetime. The bigger mean value denotes the longer time interval of applications' arrival. As shown in Fig. 4, the system lifetime increased for all profiles when the mean value is increased. However, the system lifetime for all profiles starts with and ends up with similar values in the figure. In order to reveal the reason for this system lifetime behavior, we plotted Fig. 5 to review how many applications (more precisely, the total length of applications) were performed on the system. When the mean value is small, the maximum amount of applications can be finished in the system no matter which profile is used. This is a consequence of the extensive use of data sharing among applications. Even for the precision-based profile, the overwhelming tasks make the same service to the same sensor in a short time period and leads to "passive" data sharing on that sensor. For the lifetime-base profile, SACHSEN is intended to look for the sensors which can provide the maximum time period of data sharing, and this allocation strategy contributes a lot for handling more incoming applications. This shows us, that even if the system has similar lifetime, the number of completed applications can be very different. With the increasing of the mean value, the total number of applications that a system can finish significantly decreases. This is because the possibility of

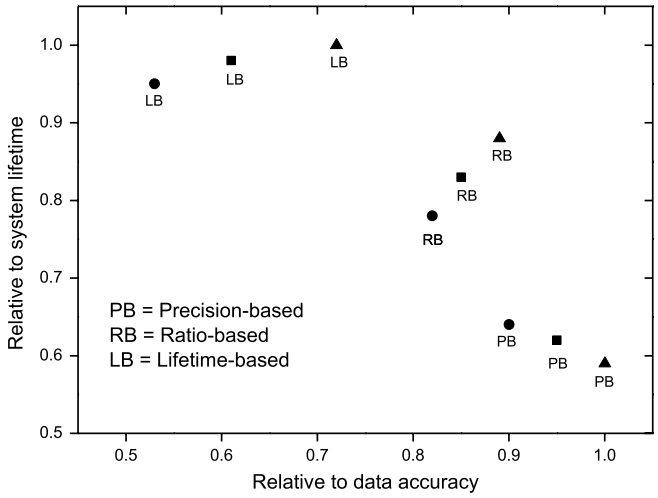


Fig. 6. Data accuracy for user-requirements profiles depending on the percentage of WSNs overlapping — small, medium, and large.

sharing data among applications is dropping. When the mean value increases to 50, there is very little chance to find out a time period for tasks to share data. The energy difference is mainly affected by routing hops of the tasks among user requirements profiles. This shows us, in Fig. 3, the growth of system lifetime is not caused by the energy-saving, but by the idle time of the system nodes.

6.1.2.2. Data accuracy. In our system, data accuracy refers to whether the data is correctly gathered by the sensor or the event of interest is correctly detected. It has two requirements: it must be the right value and it must represent the value in a consistent form with all other representations of the same value. Otherwise, the data sent back to the user can be flawed and further leads to wrong decision/action. As mentioned in Eq. (2), the data accuracy is only measured by F_i. In order to measure the data accuracy provided to an application, we define the following equation:

$$DA = \sum_{i=1}^{n} F_i \tag{8}$$

where F_i is the measurement environmental noise of the selected sensor for a task, and n is the number of tasks an application contains. For the experimental result, we will use the best case as a benchmark, and the remainder cases will be normalized by the benchmark in order to clearly reveal the relationship among different user-requirements.

Fig. 6 shows the relative performance of SACHSEN of different user-requirements under different percentage of overlapping WSNs. The overlapping percentage in this experiment is set to light (10%, represented by •), medium (50%, represented by ■) and heavy (90%, represented by ▲). The x-axis is the average relative data accuracy and the y-axis is the average relative system lifetime. It allows us to evaluate the gain regarding to contradictory metrics and to highlight dominant results. Profiles located toward the top-right corner of the graph are thus preferable.

6.1.2.3. Energy consumption. To better understand how energy consumption of each task is affected by the adoption of SACHSEN, three user profiles are used to compare with the benchmark, which is the energy consumption of each task without using SACHSEN under the same system. Moreover, by varying the application arrival rate, we intend to study the relationship between estimated and actual energy consumption.

Results shown in Fig. 7 clearly show the characteristics of different profiles and the superior performance of our algorithm compared to the benchmark, especially when the application

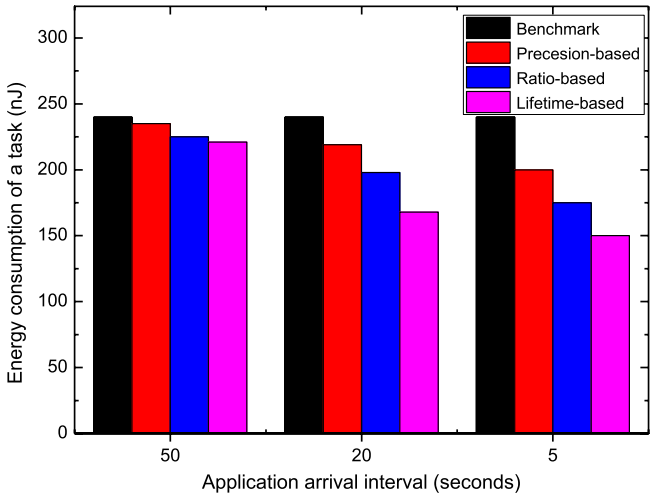


Fig. 7. Energy consumption of each task for different user profiles under different application arrival interval.

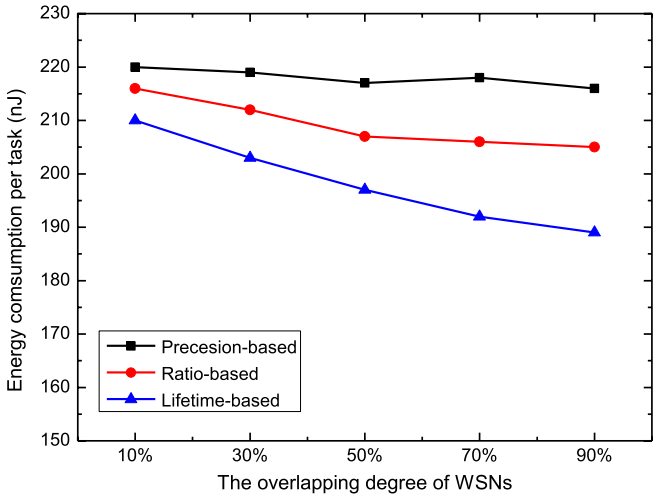


Fig. 8. Energy consumption of each task for different user profiles under different WSNs overlapping.

arrival interval is small. The performance of benchmark is not affected by the change of application arrival interval, since no SACHSEN algorithm is employed in the resource allocation process to utilize data sharing to reduce energy consumption. In other words, the energy consumption of the benchmark is the estimated value for each task. With the change of application arrival interval, the energy consumption when using the three profiles considered in SACHSEN is influenced in different degrees. Precision-based profile is the least affected since the tasks are always allocated to a sensor which is able to provide the best data accuracy. On the contrary, the lifetime-based profile can fully utilize the advantage of data sharing between tasks brought by the smaller application arrival interval. As a result, the energy consumption of each task using lifetime-based profile is the lowest one among all profiles. For the ratio-based profile, it combines the characteristics of precision-based profile and lifetime-based profile, thus its performance takes a value in the middle.

In Fig. 7, we study how the energy consumption for the different profiles is affected by application arrival interval. However, this is not the only factor which influences the algorithm performance; the overlapping degree of WSNs also affects the performance to a certain extent. This is because WSNs overlapping provides more options to SACHSEN for the task allocation, thus the algorithm operation in the different profiles can obtain different benefits from such overlapping to reduce energy consumption.

As shown in Fig. 8, the energy consumption of precision-based profile is barely affected by the overlapping degree of WSNs, since the main contribution of overlapping is to reduce the energy consumption with communication. With the increase of overlapping, for the precision-based profile, unless a sensor can provide the better data accuracy and shorter communication path to the sink node than previous selection, the energy consumption of a task can be reduced. Otherwise, the energy consumption is not significantly affected by the degree of overlapping. The ratio-based profile and lifetime-based profile both reduce the energy consumption with the increase of WSNs overlapping. For these two profiles, SACHSEN looks for the sensors which can provide the smallest communication consumption to perform the tasks, as well as the possible data sharing opportunities to the tasks. Therefore, the energy consumption to perform a task is reduced when the degree of overlapping is increased.

6.2. Testbed experiments

In this section, we present the details of our experiments for evaluating the performance of the SACHSEN on real sensor nodes of Sun SPOT platform. Sun SPOT is a sensor platform particularly suitable for rapid prototyping of WSNs applications. Sun SPOTs run a small footprint Java virtual machine that enables the nodes to be programmed using the high-level Java programming language (Java Micro Edition CLDC10). Each Sun SPOT consists of sensing devices (light, temperature, accelerometer, etc.), actuators (digital outputs, LEDs, etc.) and other essential components (radio, battery). The Sun SPOT Software Development Kit (SDK) environment contains two important tools for managing the software deployed in the SPOTs: SPOT Manager and Solarium. SPOT Manager is a WebStart application used for installing and managing the user SDK, and Solarium is a host workstation application used for managing application software on Sun SPOTs. Solarium contains a SPOT emulator that is useful for testing SPOT software and/or for creating large scale scenarios when the users have not enough real sensor nodes. In the following experiments, we use both real sensor nodes and the emulated sensor nodes generated from Solarium to evaluate the algorithm performance. For better inheritance, most experimental settings are followed the previous settings used in the simulation section. But it is worthy to note that we used Sun SPOT API to directly measure the energy consumption instead of using the adopted energy consumption equations.

6.3. Experimental result

We employed another three task allocation schemes, UMADE proposed in [2], FRG (fractional relaxation greedy) proposed in [37] and random approach as references to study the SACHSEN performance. UMADE is a QoS-aware algorithm, which is a greedy approach designed to maximize a single objective — quality of monitoring, in the task allocation. Besides that, the utilization of free memory for each sensor node is considered as an additional selection criteria. In other words, if a sensor node could provide the maximum increment in system utility per unit memory consumption, this sensor node is preferred to be used to perform the next available task. FRG is also a QoS-aware algorithm, which is a local search algorithm for allocating a certain type of networked sensing applications in WSNs in order to maximize the overall Quality of Monitoring (QoM) subject to resource constraints (e.g. memory and network bandwidth). This approach exploits the submodular property of the variance reduction functions in networked sensing applications since sensor readings from different nodes are often correlated. The semi-random approach follows the beginning stage of SACHSEN to generate the available

sensor candidates list for tasks. However, this approach is a non-QoS-aware design, which means it does not address the QoS requirements of applications. When the sensor candidates list is generated, the best candidate is simply selected from the list in a random way to perform the next available task.

In the following tests, we first evaluate the scalability of all four task allocation algorithms, namely UMADE, FRG, semi-random and SACHSEN in ratio-based mode, since we expect the overall system utility can be increasingly improve before reaching the peak value when new applications are continually arrived. All four algorithms are tested in the same scenario with the same settings. The input is set as follow. The system starts running with 2 newly arrival applications. In each pre-set time interval (30 s in our case), another 2 applications will arrive the system, until there are totally 10 applications simultaneously running within the system. The same set of applications is consecutively arrived at the same mode until one of sensor nodes within the system is run out of its energy. The applications were randomly generated as the approach described in [31]. The result is shown in Fig. 9, and it shows that as the number of WSNs increases, the system lifetime of all algorithms is increased as well. We can also observe that SACHSEN nearly performs as good as the semi-random algorithm to achieve desired system lifetime when the number of WSNs is increased. The relative performance difference between SACHSEN and semi-random algorithm is mainly affected by the algorithm design purpose and the parameters setting. As aforementioned, the semi-random algorithm is designed as a non-QoS-aware approach, the sensor nodes selection does not need to take the tradeoff between energy consumption and data accuracy of performing tasks into consideration. Besides that, in this test, we set the SACHSEN into ratio-based mode and the coefficient for two QoS parameters (energy and data accuracy) is set to 0.5 for each. Under this circumstance, SACHSEN has to sacrifice a certain amount of energy to obtain better data accuracy for some tasks to meet the QoS requirements. If the SACHSEN is set into lifetime-based mode, or even the energy QoS parameter is assigned a larger value than 0.5, we have reasons to believe that SACHSEN cloud provide superior system lifetime than the random algorithm does. Another obvious result we can conclude from the Figure is that the UMADE and FRG algorithm do not work well in the large scale WSNs, especially for the UMADE. This is due to the UMADE and FRG algorithms are both designed to maximize the quality of monitoring, as well as the system utility. The energy consumption of sensor nodes is not fully considered in the algorithm, which result in the utilization of the active sensor nodes is dropped. In other words, the incremental system utilization of UMADE is achieved by putting as many as possible sensor nodes into active mode instead of increasing each sensor node's utilization. As a result, the system lifetime of UMADE approach is not improved significantly when the number of WSNs is increased. Compared to UMADE, FRG exploits the submodular property which indirectly constraints the energy consumption since allocating a new node to a specific task could result in decreasing improvement to the QoM as the set of nodes allocated to the application grows.

In Fig. 10, the results of accuracy for all four schemes are presented, where accuracy denotes the ratio of the number of tasks being correctly executed, in other words, whether a sensor node within the system is capable of performing the specific task, compared to the total number of tasks. UMADE, SACHSEN and FRG have the similar performance and they all clearly performed better than the semi-random algorithm at all times since the semi-random algorithm does not take data accuracy into account when a sensor node is selected to perform a specific task. As shown in the Figure, the performance of SACHSEN is close to the UMADE, especially when the number of applications continuously increased, in our case, not less than 6 applications are simultaneously executing in the system. This result reveals that SACHSEN not only dynamically adjusts the QoS requirements for all the tasks in an at least best-effort manner, but also well utilizes the sensor nodes with executing tasks as a possible better

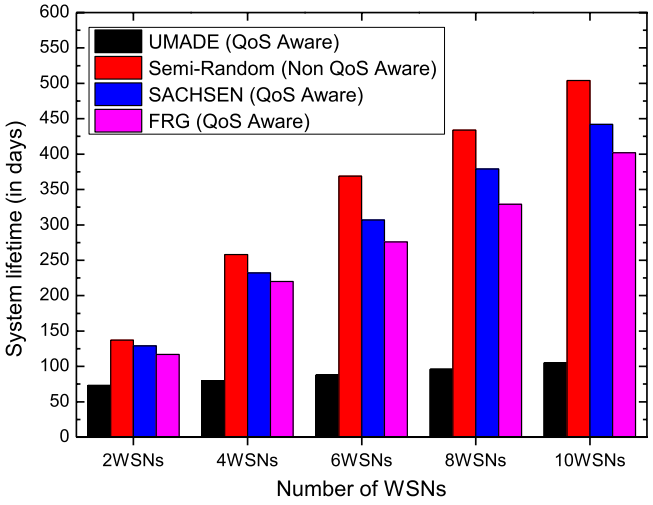


Fig. 9. System lifetime in days for all four allocation schemes.

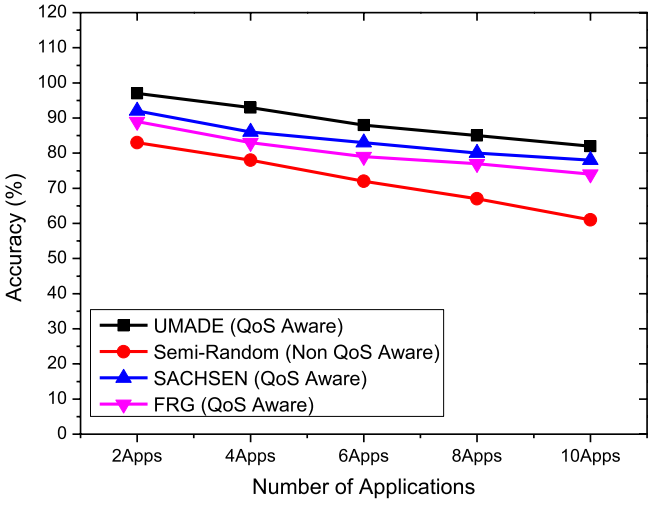


Fig. 10. Accuracy in percentage for three task allocation schemes.

choice for the await tasks while saving energy to prolong the system lifetime. We also notice that SACHSEN always outperforms FRG since SACHSEN is clearly beneficial from the task allocation decisions exploiting the latest global information instead of the limited available local information.

Finally, we perform a test to analyze how the packet loss could possibly affect the performance of the algorithms in a noisy wireless communication environment, since it is well-known that WSNs have chance to be deployed, and work in a harsh environment. The results are shown in Fig. 11. We can easily perceive that the accuracy of all four algorithms is continually dropped when the packet loss rate increases. The performance of UMADE scheme is least affected by the packet loss due to it tries to deliver the task to one of sensor nodes in the candidate sensors list according to the selection criteria unless all of them are not available. FRG adopted the similar strategy and its performance is pretty close but slightly worse than UMADE, especially when packet loss rate reaches 70%. Compared to the UMADE, the semi-random algorithm adopted the simplest mechanism to deliver the task, which means once the sensor node is randomly selected; the task is sent out straightaway without considering the result. This makes the random algorithm is very prone to the wireless communication environment. As we

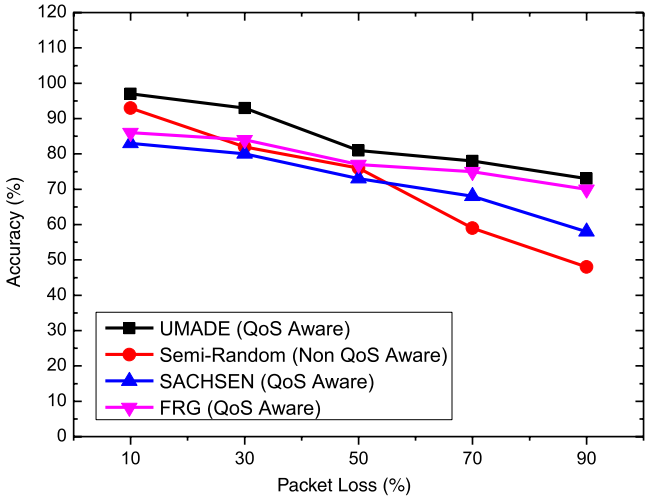


Fig. 11. Accuracy in percentage for four task allocation schemes under communication packet loss environment.

can observe from the Fig. 11, especially when the packet loss rate reaches 50%, the performance of the random algorithm is dropped dramatically compared to the other two algorithms. The performance of SACHSEN appears to be relatively stable when the packet loss rate is consecutively increased, since the best sensor candidate is chosen in advance. This mechanism requires the up-to-date sensor node information is collected regularly. If any eligible sensor nodes information is lost on the way to the sink, those nodes are automatically neglected to be sensor candidates for performing the awaiting tasks. Packet loss impact is therefore reduced to a certain extent. However, SACHSEN does not currently involve any failover mechanism to recover the pack loss from sink to sensor node. The performance of SACHSEN is inevitably reduced as the packet loss rate is increased.

7. Conclusion

In this paper, we address task allocation of multiple applications running on top of systems comprised of heterogeneous WSNs integrated to the Web. We developed a new resource allocation algorithm called SACHSEN that exploits resource heterogeneity to make effective task-sensor assignments explicitly taking into account performance requirements of application. Experimental results show that SACHSEN produces promising results for different scenarios in terms of both application performance and energy efficiency of WSNs.

In future work, we will extend SACHSEN to consider the case of partial meetings of application requests and the several available compositions. In such case, each task contributes with a different QoS value to the application and also consumes different amount of network resources. SACHSEN would need to decide which tasks have to be completed to meet the minimum requirements of applications, while maximizing the system utility for all the applications.

Acknowledgments

Dr. Li's work was supported by Australian Postgraduate Award (APA) and NICTA enhanced PHD program. Prof. Delicato and Prof. Pires' works were supported by Brazilian Funding Agencies FAPERJ and CNPq (under grants 470586/2011-7 and 311363/2011-3 for Flávia Delicato and 310661/2012-9 for Paulo Pires). Professor Zomaya's work was supported by the Australian Research Council Discovery Grant (DP1097111).

References

[1] L. Atzori, A. Iera, G. Morabito, The internet of things: a survey, Computer Networks 54 (15) (2010) 2787–2805.
[2] S. Bhattacharya, et al., Multi-application deployment in shared sensor networks based on quality of monitoring, in: 16th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS), 2010.
[3] J. Byers, G. Nasser, Utility-based decision-making in wireless sensor networks, in: First Annual Workshop on Mobile and Ad Hoc Networking and Computing, MobiHOC, 2000.
[4] H. Chen, M. Maheswaran, Distributed dynamic scheduling of composite tasks on grid computing systems, in: Proceedings of the 16th International Parallel and Distributed Processing Symposium, IEEE Computer Society, 2002, p. 119.
[5] D. Chen, P.K. Varshney, QoS support in wireless sensor networks: a survey, in: Proceedings of the 2004 International Conference on Wireless Networks (ICWN2004), Las Vegas, Nevada, USA, June, 2004.
[6] Corporation, O. Sun SPOT World. 2012 [cited 2012. Available from: http://www.sunspotworld.com/.
[7] P.J. del Cid, et al., Middleware for resource sharing in multi-purpose wireless sensor networks, in: IEEE International Conference on Networked Embedded Systems for Enterprise Applications (NESEA), 2010.
[8] F. Delicato, et al., An efficient heuristic for selecting active nodes in wireless sensor networks, Computer Networks 50 (18) (2006) 3701–3720.
[9] F.C. Delicato, J.M.T. Portocarrero, J.R. Silva, P.F. Pires, R.P.M.de Araújo, T. Batista, MARINE: MiddlewAre for resource and mIssion-oriented sensor NEtworks, ACM SIGMOBILE Mobile Computing and Communications Review 17 (1) (2013) 40–54.
[10] I. Dietrich, F. Dressler, On the lifetime of wireless sensor networks, ACM Transactions on Sensor Networks (TOSN) 5 (1) (2009) 1–39.
[11] R.T. Fielding, Architectural Styles and the Design of Network-based Software Architectures, University of California, 2000.
[12] C.-L. Fok, G.-C. Roman, C. Lu, Adaptive service provisioning for enhanced energy efficiency and flexibility in wireless sensor networks, Science of Computer Programming (0) (2011).
[13] D. Guinard, V. Trifa, Towards the web of things: web mashups for embedded devices, in: Proceedings of Workshop on Mashups, Enterprise Mashups and Lightweight Composition on the Web, International World Wide Web Conferences, 2009. Madrid, Spain, 2009.
[14] V.C. Gungor, L. Bin, G.P. Hancke, Opportunities and challenges of wireless sensor networks in smart grid, IEEE Transactions on Industrial Electronics 57 (10) (2010) 3557–3564.
[15] W.B. Heinzelman, A.P. Chandrakasan, H. Balakrishnan, An application-specific protocol architecture for wireless microsensor networks, IEEE Transactions on Wireless Communications 1 (4) (2002) 660–670.
[16] Z. Henan, R. Sakellariou, Scheduling multiple DAGs onto heterogeneous systems, in: 20th International Parallel and Distributed Processing Symposium, IPDPS, 2006.
[17] M. Kushwaha, et al., OASiS: a programming framework for service-oriented sensor networks, in: 2nd International Conference on Communication Systems Software and Middleware, COMSWARE, 2007.
[18] E.A. Lee, Cyber physical systems: design challenges, in: 11th IEEE International Symposium on Object Oriented Real-Time Distributed Computing (ISORC), 2008.
[19] W. Li, F.C. Delicato, A.Y. Zomaya, Adaptive energy-efficient scheduling for hierarchical wireless sensor networks, ACM Transactions on Sensor Networks (TOSN) 9 (3) (2013) 1–34.
[20] T. Liu, et al., Implementing software on resource-constrained mobile sensors: experiences with Impala and ZebraNet, in: Proceedings of the 2nd International Conference on Mobile Systems, Applications, and Services, ACM, Boston, MA, USA, 2004, pp. 256–269.
[21] E. Loureiro, et al., A flexible middleware for service provision over heterogeneous pervasive networks, in: International Symposium on a World of Wireless, Mobile and Multimedia Networks, WoWMoM, 2006.
[22] A. Mainwaring, et al., Wireless sensor networks for habitat monitoring, in: Proceedings of the 1st ACM International Workshop on Wireless Sensor Networks and Applications, ACM, Atlanta, Georgia, USA, 2002, pp. 88–97.
[23] N. Mohamed, J. Al-Jaroodi, A survey on service-oriented middleware for wireless sensor networks, Service Oriented Computing and Applications 5 (2) (2011) 71–85.
[24] T. Mullen, V. Avasarala, D.L. Hall, Customer-driven sensor management, IEEE Intelligent Systems 21 (2) (2006) 41–49.
[25] T. N'Takpe, F. Suter, Concurrent scheduling of parallel task graphs on multi-clusters using constrained resource allocations, in: Proceedings of the 2009 IEEE International Symposium on Parallel & Distributed Processing, IEEE Computer Society, 2009, pp. 1–8.
[26] H. Rowaihy, et al., A survey of sensor selection schemes in wireless sensor networks, SPIE Unattended Ground, Sea, and Air Sensor Technologies and Applications IX, 2007.
[27] H. Rowaihy, et al., Sensor-mission assignment in wireless sensor networks, ACM Transactions on Sensor Networks (TOSN) 6 (4) (2010) 1–33.
[28] N. Roy, V. Rajamani, C. Julien, Supporting multi-fidelity-aware concurrent applications in dynamic sensor networks, in: 8th IEEE International Conference on Pervasive Computing and Communications Workshops (PERCOM Workshops), 2010.

[29] D.C. Schmidt, et al., Pattern-oriented Software Architecture: Patterns for Concurrent and Networked Objects, Vol. 2, Wiley, 2000.
[30] A. Scholz, et al., eSOA — Service Oriented Architectures adapted for embedded networks, in: 7th IEEE International Conference on Industrial Informatics, INDIN, 2009.
[31] X. Shuguang, et al., Multiple task scheduling for low-duty-cycled wireless sensor networks, in: INFOCOM, 2011 Proceedings IEEE, 2011.
[32] H. Sohn, et al., An experimental study of temperature effect on modal parameters of the Alamosa Canyon bridge, Earthquake Engineering & Structural Dynamics 28 (8) (1999) 879–897.
[33] C. Szyperski, Component Software: Beyond Object-Oriented Programming, Addison-Wesley Professional, 2011, p. 624.
[34] Y. Tian, E. Ekici, Cross-layer collaborative in-network processing in multihop wireless sensor networks, IEEE Transactions on Mobile Computing (2007) 297–310.
[35] L. Wei, R. Vesilo, Modeling of session persistence in web server systems, in: Australian Telecommunications Networks and Application Conference, Melbourne, 2006.
[36] G. Werner-Allen, et al., Fidelity and yield in a volcano monitoring sensor network, in: Proceedings of the 7th Symposium on Operating Systems Design and Implementation, USENIX Association, Seattle, Washington, 2006, pp. 381–396.
[37] Y. Xu, et al., Near optimal multi-application allocation in shared sensor networks, in: Proceedings of the Eleventh ACM International Symposium on Mobile Ad Hoc Networking and Computing, ACM, Chicago, Illinois, USA, 2010, pp. 181–190.
[38] A.Y. Zomaya, Parallel and Distributed Computing Handbook, McGraw-Hill, Inc., 1996.



Wei Li received his Ph.D. degree from the School of Information Technologies at the University of Sydney in 2012. He is currently a research associate in the Centre for Distributed and High Performance Computing, University of Sydney. His current research interests include wireless sensor network, Internet of Things, task scheduling, nature-inspired algorithms, and optimization. He is a member of IEEE.



Flávia C. Delicato received her Master Degree (2000) and Ph.D. (2005) from Federal University of Rio de Janeiro. In 2010 she was a visiting academic in a post-doctoral stage at the University of Sydney, Australia. She is currently an Associate Professor of the Department of Computer Science at the Federal University of Rio de Janeiro, Brazil where she teaches for undergraduate and post-graduate courses, and integrates the Computer Networks and Distributed Systems Research Group and the Laboratory for Ubiquitous Computing (http://ubicomp.nce.ufrj.br/ubicomp/?lang=en). She is a Researcher Fellow of the National Council for Scientific and Technological Development (CNPq) and a Young Researcher Fellow from FAPERJ Brazilian Funding Agency. She has published more than 100 refereed international conference and journal papers. She is an Editorial Board Member of The Brazilian Journal of Computer Networks and Distributed Systems and of the International Journal of Computer Networks (IJCN). She has been participating in several research projects with funding from International and Brazilian government agencies, including CNPq (National Council for Scientific and Technological Development), CAPES (Coordenação de Aperfeiçoamento de Pessoal de Nível Superior), ANP (Agência Nacional do Petróleo, Gás Natural e Biocombustíveis), RNP (Rede Nacional de Ensino e Pesquisa), Fundación Carolina (Spain), FAPERJ and AusAID (from the Australian Government). She integrates the Centre for Distributed and High Performance Computing of the University of Sydney (http://sydney.edu.au/distributed_computing/). Her main research interests are: Wireless Sensor Networks, Ubiquituos Computing; Adaptive Middleware; Model Driven Development; Web Service Technologies, WoT and IoT.



Paulo F. Pires is currently Associate Professor at the Department of Computer Science at Federal University of Rio de Janeiro (UFRJ), Brazil and member of the Centre for Distributed and High Performance Computing, University of Sydney, Australia. His main research interests include model driven development, software product lines, infrastructures for Web service composition, and application of Software Engineering techniques in the development of software systems for emerging domains, such as embedded, ubiquitous and pervasive systems. Dr. Pires has held a technological innovation productivity scholarship from the Brazilian Research Council (CNPq) since 2010 and is a member of the Brazilian Computer Society (SBC).



Young Choon Lee received the B.Sc. (hons) degree in 2003 and the Ph.D. degree from the School of Information Technologies at the University of Sydney in 2008. He is currently a postdoctoral research fellow in the Centre for Distributed and High Performance Computing, School of Information Technologies. His current research interests include scheduling and resource allocation for distributed computing systems, nature-inspired techniques, and parallel and distributed algorithms. He is a member of the IEEE and the IEEE Computer Society.



Albert Y. Zomaya is currently the Chair Professor of High Performance Computing & Networking in the School of Information Technologies, The University of Sydney. He is the author/co-author of seven books, more than 400 papers, and the editor of nine books and 11 conference proceedings. He is the Editor in Chief of the IEEE Transactions on Computers and serves as an associate editor for 19 leading journals. Professor Zomaya is the recipient of the IEEE TCPP Outstanding Service Award and the IEEE TCSC Medal for Excellence in Scalable Computing, both in 2011. He is a Chartered Engineer, a Fellow of AAAS, IEEE, IET (UK).



Claudio Miceli de Farias received a M.Sc. degree in Computer Science in 2010 from the Federal University of Rio de Janeiro, Brazil. His research interests include smart grids, ambient intelligence, wireless sensor networks, network security, VOIP, real-time communications and video processing.



Luci Pirmez is a professor at the Institute of Informatics of the Federal University of Rio de Janeiro (UFRJ), Brazil. She received her M.Sc. and Ph.D. degree, both in Computer Science from the Federal University of Rio de Janeiro, Brazil in 1986 and 1996, respectively. She is a member of research staff of the Computer Center of Federal University of Rio de Janeiro. Her research interests include wireless networks, wireless sensor networks, network management and security. She is one of 300 researchers in Computer Science from all over Brazil selected to be CNPq researchers. She is currently involved in a number of research projects with funding from Brazilian government agencies, in the areas of wireless networks, wireless sensor networks, network management and security.